(continued)

As the error says, running aggregations on a `text` field is prohibited by default, so if we want to perform aggregations on `text` fields, we need to enable `fielddata` on the respective fields. We can set `"fielddata": true` when defining the mapping:

```
PUT tv_sales_with_field_data
{
  "mappings": {
    "properties": {
      "name":{
        "type": "text",
        "fielddata": true
      }
    }
  }
}
```

Note that enabling `fielddata` may lead to performance problems because data is stored in memory on the nodes. Rather than taking a performance hit by enabling `fielddata`, we can instead create a multi-field data type with `keyword` as the second type. This works because `keyword` data types are allowed for aggregations.

Finding the average of a set of numbers is a frequent operation in analytics. As expected, Elasticsearch provides a handy function called `avg` to find averages, which is the topic of the next section.

13.2.3 The avg metric

Finding the average of a set of numbers is a basic statistical function that we often require. Elasticsearch provides the `avg` metric aggregation out of the box for running an average calculation. For example, the next query fetches the average TV price using `avg`.

Listing 13.3 Average price of all TVs

```
GET tv_sales/_search
{
  "size": 0,
  "aggs": {
    "tv_average_price": {
      "avg": {
        "field": "price_gbp"
      }
    }
  }
}
```

Names the aggregation

Calculates the average price

Field on which the average is calculated

`tv_average_price` is a user-defined name given to this average aggregation. The `avg` declaration in the code represents the average function. The data field on which we